Trabajo Práctico Integrador

Lector de texto en imágenes con un modelo de Hugging Face

Materia: Seminario de Actualización
Alumna: Natalia Nodar
Docente: Cynthia M. Villagra

A. El problema

1. ¿Qué problema querés resolver y para quién? ¿Por qué un modelo de IA y no unas reglas escritas a mano?

Me basé en un problema que tenia en el trabajo, detectando manuscritos en imágenes que contuvieran datos sensibles. Si bien utilizábamos un OCR de Google, no era bueno transcribiendo manuscritos, entonces me propuse buscar algún modelo, que leyera español y fuera bueno detectando textos en imágenes. Esto tiene uso tanto para extraer textos que luego sean leídos por un modelo de prevención de fraude más liviano, o bien para extraer fotos de apuntes de clase, como los ejemplos que usé de prueba.

Un if con palabras clave no sirve para esto, porque trabaja sobre texto que ya existe, y acá hay que reconocer las letras a partir de los píxeles de la imagen. Cada tipografía, cada letra manuscrita, la inclinación de la foto, la luz o el ruido cambian por completo los píxeles de una misma palabra, y no hay forma de escribir reglas para todos esos casos.

B. El modelo

2. ¿Qué modelos consideraste y por qué te quedaste con uno?

Comparé dos modelos de la tarea image-text-to-text. Los datos salen de sus *model cards* en Hugging Face:

	LightOnOCR-2-1B (elegido)	Qwen3-VL-4B-Instruct
Tipo	Especialista en OCR: solo transcribe documentos	Generalista de visión y lenguaje: también describe imágenes y responde preguntas
Tamaño de los pesos	~1.000 millones de parámetros · 2,0 GB (BF16)	~4.400 millones de parámetros · 8,9 GB (BF16)
Idiomas	11, con más cobertura de francés	OCR en 32 idiomas
Datos de entrenamiento	Dataset propio LightOnOCR-mix-0126 (documentos escaneados, arXiv) y ajuste con RLVR	Corpus multimodal general; la card destaca robustez con poca luz, desenfoque e inclinación
Entrada	Solo la imagen	Imagen + instrucción de texto
¿Menciona manuscritos?	No	No

También miré microsoft/trocr-base-handwritten, especializado en letra manuscrita, que es lo que buscaba, pero lo descarté porque está entrenado solo con texto en inglés, lee un renglón por vez y ningún proveedor lo sirve, así que no podía usarlo con InferenceClient.

Probé los dos modelos con 8 fotos mías. Cinco son casos difíciles, pensados para que el modelo se equivoque. Para cada foto anoté 2 a 4 palabras clave visibles para comparar la precisión lectura de cada uno.

Me quedé con LightOnOCR-2-1B porque, si bien tiene una precisión un poco menor, pesa 4 veces menos (2 GB contra casi 9 GB) y es más rápido. Lo que resigno es que Qwen lee un poco mejor manuscritos, pero el costo vs. beneficio de tiempos y valor de la consulta decantaron en mi decisión.

3. ¿Dónde se equivoca?

El puntaje mostró pocos errores, pero al leer el texto completo aparecieron fallas que las palabras clave no detectan:

Imagen	LightOnOCR-2-1B	Qwen3-VL-4B
Folleto en perspectiva	"Planta" por *Pisos*, "Buc Hombres" por *Machimbres*	Casi perfecto
Revista inclinada	Repite "Claves para construir con madera"	Inventa nombres en créditos ilegibles
Libro en inglés borroso	Título bien; el resto, como la calidad de la imagen es mala, inventa	Título bien; el resto, como la calidad de la imagen es mala, inventa
Tabla manuscrita	La arma como tabla, con filas y columnas correctas	La lee por filas y traduce "REPLICACIÓN" a *REPLICATION*
Apuntes en cursiva	"Ayedrez" por *Ajedrez*	Bien
Ticket	Cambia el dominio: "supermercedasdia.com.br"	Perfecto

LightOnOCR se entrenó con documentos escaneados y PDFs, y su card no menciona letra manuscrita: por eso estructura muy bien tablas y títulos, pero confunde letras en lo escrito a mano. Qwen se entrenó con imágenes mucho más variadas, y por eso tolera mejor la letra a mano y las fotos torcidas.

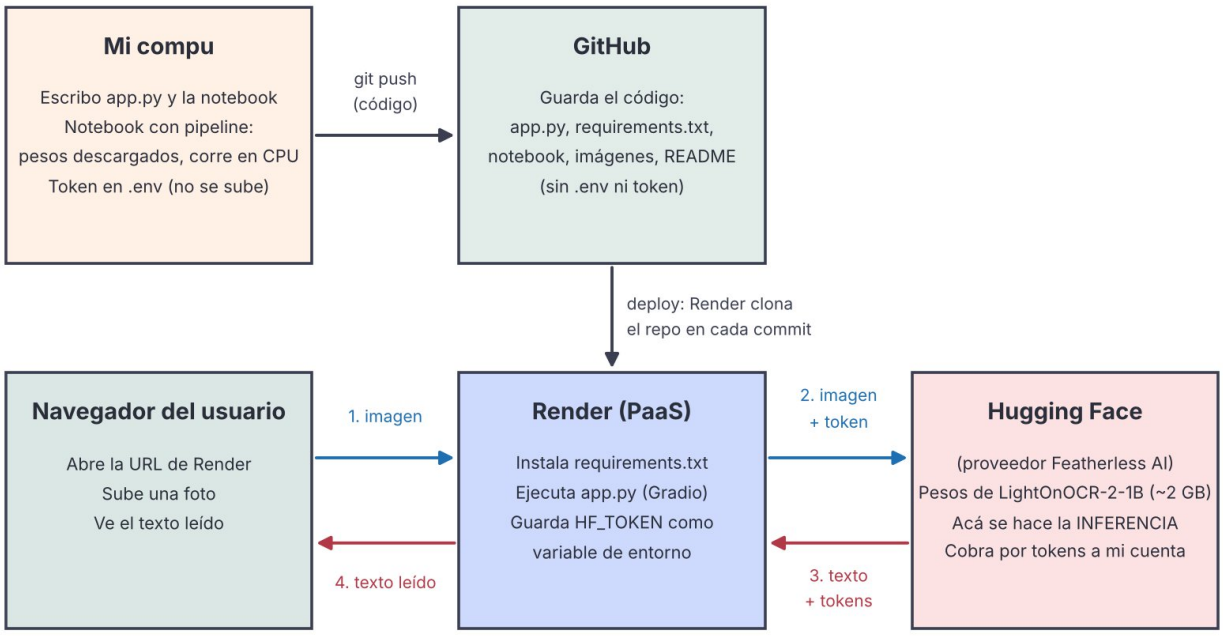
El error más importante es que los dos inventan texto cuando la imagen no se puede leer, en lugar de avisar.

4. Tu app, ¿está entrenando o infiriendo?

Está infiriendo. Uso un modelo que ya entrenó la empresa LightOn, sin cambiar ninguno de sus pesos: le paso una imagen y me devuelve el texto. En mi app los pesos son siempre los mismos: cada foto que sube un usuario es una predicción nueva con el mismo modelo, y el modelo no aprende nada de ella.

C. Dónde corre cada cosa

5. ¿Dónde vive el código y dónde vive el modelo?



- El código lo escribo en VS de manera local y lo guardo en GitHub. Render lo clona y lo ejecuta (app.py).
- Los pesos del modelo (~2 GB) están en Hugging Face y en los servidores del proveedor Featherless AI. Mi app nunca los descarga.
- La inferencia se hace en los servidores del proveedor. Render solo manda la imagen y recibe el texto.
- El token vive en mi archivo .env (que no se sube a GitHub) y, en el deploy, como variable de entorno de Render.
- En el notebook fue distinto: con pipeline, los pesos se descargaron a mi compu y la inferencia corrió en mi procesador.

6. ¿Usaste pipeline o InferenceClient en la app publicada?

Usé InferenceClient por los tiempos de respuesta (mas detalle ver notebook).

Con pipeline, el servidor de Render tendría que descargar y cargar 2 GB de pesos además de torch, y el plan gratuito no tiene esa memoria. Aun en un servidor más grande, cada imagen tardaría más de un minuto y medio en CPU. Con InferenceClient la app es liviana (su requirements.txt ni siquiera incluye torch) y responde en segundos.

Acá, para poder comparar tiempos, decidí hacer una carga mínima de $5 usd para usar en créditos en Hugging Face. De lo contrario, estaría obligada a usar pipeline en Colab.

D. El deploy

7. ¿Dónde deployaste y por qué ahí?

Publiqué la app en Render, que es un PaaS (plataforma como servicio). Se conecta a mi repositorio de GitHub: le indico la carpeta TP_Integrador, el comando de instalación (pip install -r requirements.txt) y el de inicio (python app.py), y Render se encarga del servidor, del HTTPS y de la URL pública. Además me deja guardar el token como variable de entorno, fuera del código. El plan gratuito alcanza porque mi app es liviana: no corre el modelo porque uso los créditos de Huggingface, fue mi decision para poder probar escenarios.

Opciones que descarté:

- **Github pages** (que usé en otro trabajo para otra materia): solo sirve archivos HTML o PHP y no ejecuta Python. Además, si llamara a Hugging Face desde el navegador, el token quedaría a la vista de cualquiera.

- **VPS:** me daría control total, pero tendría que instalar y mantener yo el sistema operativo, Python, la seguridad y el reinicio de la app. Es mucho trabajo para una app chica, y es pago.

- **Serverless:** son funciones que se prenden con cada pedido y se apagan enseguida, con tiempos máximos cortos. Gradio, en cambio, es un servidor que tiene que quedar escuchando y mantener la conexión con el navegador mientras el modelo responde.

- **Nubes elásticas** (AWS, Google Cloud, Azure): sirven para escalar a miles de usuarios, pero su configuración es compleja y los costos son difíciles de controlar para pruebas pequeñas como las que hice.

Mi app necesita un servidor que quede encendido porque es un proceso de Python que tiene que estar escuchando en un puerto para recibir fotos en cualquier momento. Además, el token, por seguridad tiene que vivir en el servidor y no en el navegador. En el plan gratuito, Render apaga el servicio después de 15 minutos sin visitas y tarda cerca de un minuto en volver a arrancar. Para que esté siempre disponible, configuré un servicio externo (cron-job.org) visita la URL cada 10 minutos.

E. Lo que se rompio

8. Errores reales que aparecieron durante el trabajo

Error 1 · LightOnOCR me traía texto sin sentido

```
pd.set_option("display.max_colwidth", 120)
columnas = ["imagen", "¿difícil?", "texto leído", "palabras clave encontradas", "puntaje", "¿acertó?", "segundos"]

print("MODELO A ·", MODELO_A)
display(res_a[columnas])
```

0.0s

```
MODELO A · lightonai/LightOnOCR-2-1B
```

	imagen	¿difícil?	texto leído	palabras clave encontradas	puntaje	¿acertó?	segundos
0	Impreso: tapa de manual		عثعثعثع

Error 2 · 401 Unauthorized · Invalid username or password

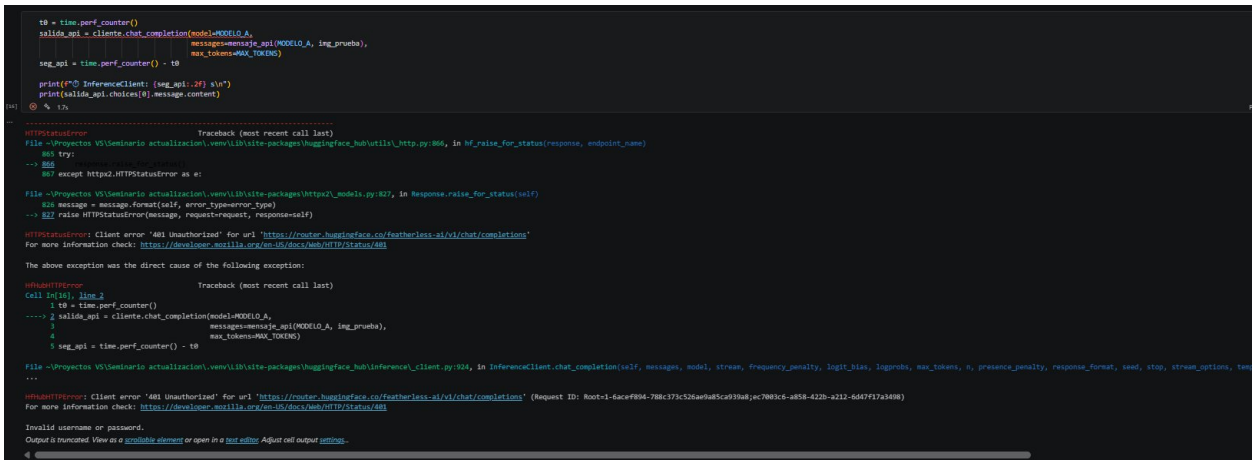


Figura 4. El error en el notebook (mi compu).

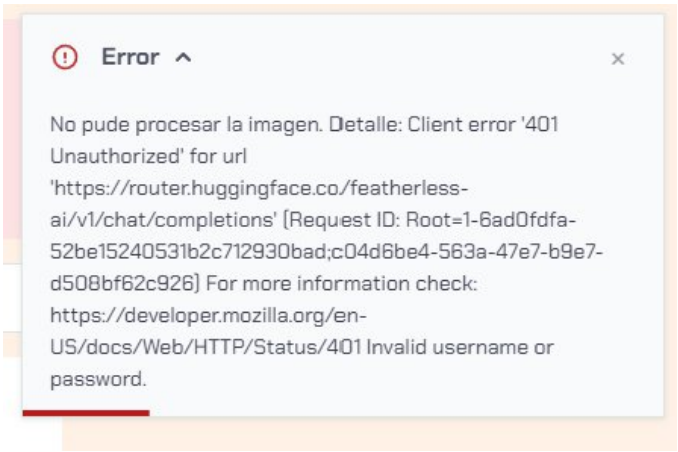


Figura 5. El mismo error en la app ya publicada en Render.

Dónde apareció: en Hugging Face, al llamar con `InferenceClient`: primero desde el notebook y después desde la app publicada en Render.

Qué lo causaba: Hugging Face rechazaba el token. En el notebook, el `.env` tenía un token que yo había revocado, y aunque lo cambié, `load_dotenv()` no reemplazaba la variable que el kernel ya tenía cargada. En Render, cargué en *Environment* el mismo token viejo.

Cómo lo resolví:

1. Crear un token nuevo en Hugging Face (*Settings → Access Tokens*) con el permiso "Make calls to Inference Providers".

2. Pegarlo en el `.env` y usar `load_dotenv(override=True)`, para que el archivo le gane a la variable vieja.

3. Pegar en Render el token de nuevo y corroborando que coincidera con el de `.env`.

Error 3 · OSError: Cannot find empty port in range: 7860-7860



Figura 6. Error al correr app.py en mi compu.

Dónde apareció: en mi compu, al ejecutar python app.py.

Qué lo causaba: otra app de Gradio (de una clase anterior) seguía ocupando el puerto 7860, que es el que Gradio usa por defecto. Windows no deja que dos programas usen el mismo puerto.

Cómo lo resolví:

1. Cambiar app.py para que, si no existe la variable PORT, use server_port=None: así Gradio prueba desde el 7860 en adelante y toma el primero libre.

2. En Render no cambia nada: la plataforma define PORT y la app escucha en ese puerto.

Otros errores que también aparecieron ni bien arranqué las pruebas (No saqué captura): 402 Payment Required ("You have no remaining credits"), porque mi cuenta no tenía créditos para usar InferenceClient; lo resolví cargando US$ 5 y fijando un límite de gasto de US$ 5. Y model_not_supported, porque el modelo lo sirve un solo proveedor y, sin créditos, el router no encontraba a quién mandar el pedido; lo resolví indicando el proveedor con InferenceClient(provider="featherless-ai", ...).

9. Límites de tu solución

- **Si la usan 1000 personas a la vez:** el plan gratuito de Render es un único servidor chico que no escala. Los pedidos se acumularían en cola y la app se volvería muy lenta o se caería. Además, cada imagen gasta unos 1.700 tokens: con mil usuarios se acabaría rápido mi saldo de US$ 5 (configuré un límite de gasto en US$ 5) y la app dejaría de responder.

- **Si Hugging Face cambia las condiciones:** mi app depende de un solo proveedor (Featherless AI) para este modelo. Si deja de servirlo o cambia el precio, la app deja de funcionar.

- **Para que sea un producto real cambiaría:** un plan pago de Render que no se apague y pueda sumar servidores; un límite de uso por persona; un aviso de privacidad claro, porque las imágenes salen hacia terceros; un plan B (otro proveedor, o el modelo en un servidor propio con GPU como Colab); y alguna forma de detectar cuando el modelo inventa texto.

F. Vocabulario

10. Cinco líneas de app.py

```
cliente = InferenceClient(provider=PROVEEDOR, token=HF_TOKEN)

respuesta = cliente.chat_completion(model=MODELO, messages=mensajes, max_tokens=MAX_TOKENS)

texto = limpiar(respuesta.choices[0].message.content or "")

imagen.thumbnail((LADO_MAX, LADO_MAX))

boton.click(leer_texto, inputs=entrada, outputs=[salida_texto, salida_md, detalle])
```

Concepto	Dónde está	Qué es
Clase	InferenceClient (línea 1)	El "molde" que define cómo hablar con Hugging Face.
Instancia	cliente (línea 1)	El objeto concreto que creo a partir de esa clase, con mi proveedor y mi token.
Método	chat_completion (línea 2); también thumbnail y click	Una función que pertenece al objeto y se llama con un punto: cliente.chat_completion(...).
Atributo	choices, message y content (línea 3)	Datos guardados dentro del objeto respuesta; se leen con un punto, sin paréntesis.
Parámetro y argumento	provider=PROVEEDOR (línea 1)	provider es el **parámetro** (el nombre que define la función) y PROVEEDOR ("featherless-ai") es el **argumento** (el valor que le paso).
Valor de retorno	respuesta (línea 2)	Lo que devuelve chat_completion: un objeto ChatCompletionOutput con el texto leído.

Links y uso de IA

App publicada:
https://nodarnatalia-seminarioactualizacion-1.onrender.com/?__theme=light

Carpeta del TP en el repositorio:
https://github.com/natisol87/NodarNatalia_SeminarioActualizacion/tree/main/TP_Integrador

Uso de IA generativa:
Usé Claude para ayudarme con los errores que me fueron surgiendo, asi como estructurar mejor el notebook y hacerle algunos cambios de diseño a la app. Tambien me hizo un borrador inicial para darle formato al informe, pero lo terminé editando a mano.
Las pruebas, las fotos, la ejecución y las decisiones finales son mías.